

C_NOTAS_CONTENIDO.pdf
PDF

Programación en C

en otras conversaciones me has ido enseñando como funciona C

el apartado 10 tiene 2 subapartados, en este caso, explica primero con mucha profundidad, casos particulares, miniejemplos cada una de las cosas de dicho apartado, al final incluye un código en el que se use todo

Pensó durante unos pocos segundos >

Claro. Según tus apuntes, el **apartado 10** es **Memoria dinámica**, dividido en:

1. **Petición de memoria:** `malloc`, `free`, `calloc`, `sizeof`, `realloc`, `strdup`
2. **Bloques de memoria:** `memcpy`, `memset`, `memcmp`  C_NOTAS_CONTENIDO

Voy por partes.

10. Memoria dinámica en C

En C tienes dos formas principales de guardar datos:

```
int x = 5;  
int array[10];
```

Eso es **memoria estática/local automática**. El tamaño se conoce normalmente antes o durante la ejecución de la función, y la memoria se libera sola cuando sales del bloque.

Pero a veces necesitas memoria cuyo tamaño **no conoces hasta que el programa está funcionando**.

Ejemplo:

```
int n;  
scanf("%d", &n);  
  
int array[n]; // esto existe en C moderno, pero no siempre es lo ideal
```

Para eso existe la **memoria dinámica**.

La memoria dinámica se pide manualmente usando funciones como:

```
malloc  
calloc  
realloc  
strdup
```

Y se libera manualmente con:

```
</> C
```



La idea central es:

```
</> C
```



`malloc` devuelve una dirección de memoria. Esa dirección la guardas en un puntero.

1. sizeof

Antes de `malloc`, hay que entender `sizeof`.

`sizeof` devuelve cuántos bytes ocupa un tipo o variable.

Ejemplo:

```
printf("%zu\n", sizeof(int));  
printf("%zu\n", sizeof(char)).
```

</> C



Normalmente:

```
sizeof(char)    // 1 byte  
sizeof(int)     // 4 bytes normalmente  
sizeof(double)  // 8 bytes normalmente
```

Pero no debes asumirlo siempre. Por eso se usa `sizeof`.

Ejemplo básico

```
int x = 10;  
  
printf("%zu\n", sizeof(x));  
printf("%zu\n", sizeof(int));
```

Ambos suelen imprimir `4`.

Caso importante con arrays

```
int nums[5];  
  
printf("%zu\n", sizeof(nums));
```

Si `int` ocupa 4 bytes, entonces:

</> C



Porque tienes 5 enteros:

</> C



Pero cuidado:

```
void imprimir(int nums[]) {  
    printf("%zu\n", sizeof(nums));  
}
```

Aquí `nums` realmente es un puntero, no el array completo. Entonces `sizeof(nums)` devuelve el tamaño del puntero, no del array.

2. malloc

`malloc` significa **memory allocation**.

Sirve para pedir memoria dinámica.

Su sintaxis es:

</> C



Ejemplo:

</> C



Esto pide espacio para **un entero**.

Visualmente:

</> C



Ahora puedes usarlo así:

```
*p = 25;
printf("%d\n", *p);
```

Ejemplo completo

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p = malloc(sizeof(int));

    if (p == NULL) {
        printf("Error reservando memoria\n");
        return 1;
    }

    *p = 42;

    printf("Valor: %d\n", *p);

    free(p);

    return 0;
}
```

¿Por qué comprobamos NULL ?

Porque malloc puede fallar.

Si no hay memoria suficiente, devuelve:

</> C

Entonces esto sería peligroso:

```
int *p = malloc(sizeof(int));  
*p = 5; // peligroso si p == NULL
```

Por eso se hace:

```
if (p == NULL) {  
    // error  
}
```

malloc para arrays

Si quieres un array dinámico de 5 enteros:

```
<> C
```

Esto reserva espacio para:

```
array[0]  
array[1]  
array[2]  
array[3]  
array[4]
```

Puedes usarlo casi como un array normal:

```
array[0] = 10;  
array[1] = 20;  
array[2] = 30;
```

Ejemplo:

```
int *nums = malloc(5 * sizeof(int));

if (nums == NULL) {
    return 1;
}

for (int i = 0; i < 5; i++) {
    nums[i] = i * 10;
}

for (int i = 0; i < 5; i++) {
    printf("%d\n", nums[i]);
}

free(nums);
```

3. free

Toda memoria que pides con `malloc`, `calloc`, `realloc` o `strdup` debe liberarse con `free`.

Ejemplo:

```
int *p = malloc(sizeof(int));

*p = 10;

free(p);
```

`free(p)` le dice al sistema:

Ya no necesito esta memoria.

Error grave: usar memoria después de `free`

```
int *p = malloc(sizeof(int));
*p = 10;
```

```
free(p);

printf("%d\n", *p); // MAL
```

Después de `free`, `p` sigue guardando una dirección, pero esa memoria ya no te pertenece.

A esto se le llama:

</> C



Para evitarlo, a veces se hace:

```
free(p);
p = NULL;
```



Así, si intentas usarlo, será más fácil detectar el error.

Error grave: hacer `free` dos veces

```
free(p);
free(p); // MAL
```



Esto se llama:

</> C



Puede romper el programa.

Mejor:


</> C

Porque:


</> C

es seguro. No pasa nada.

4. calloc

calloc también reserva memoria dinámica, pero con una diferencia importante:


</> C

reserva memoria sin inicializar.


</> C

reserva memoria e inicializa todo a cero.

Sintaxis:


</> C

Ejemplo:


</> C

Esto crea 5 enteros inicializados a 0.

Comparación

Con `malloc` :

</> C



Los valores iniciales son basura:

```
nums[0] // valor indeterminado  
nums[1] // valor indeterminado
```



Con `calloc` :

</> C



Los valores son:

```
nums[0] == 0  
nums[1] == 0  
nums[2] == 0  
nums[3] == 0  
nums[4] == 0
```

Ejemplo

```
int *nums = calloc(5, sizeof(int));  
  
if (nums == NULL) {  
    return 1;  
}  
  
for (int i = 0; i < 5; i++) {  
    printf("%d\n", nums[i]);  
}
```

```
}
```

```
free(nums);
```

Imprime normalmente:

```
0  
0  
0  
0  
0
```

5. realloc

`realloc` sirve para cambiar el tamaño de una zona de memoria ya reservada.

Ejemplo:

```
</> C
```



Tienes espacio para 3 enteros.

Después quieres espacio para 6:

```
</> C
```



Problema importante

No deberías hacer directamente esto:

```
</> C
```



Porque si `realloc` falla, devuelve `NULL`, y pierdes la dirección original.

Mejor:

```
int *temp = realloc(nums, nuevo_tamaño);

if (temp == NULL) {
    // nums sigue siendo válido
} else {
    nums = temp;
}
```

Ejemplo correcto

```
int *nums = malloc(3 * sizeof(int));

if (nums == NULL) {
    return 1;
}

nums[0] = 10;
nums[1] = 20;
nums[2] = 30;

int *temp = realloc(nums, 6 * sizeof(int));

if (temp == NULL) {
    free(nums);
    return 1;
}

nums = temp;

nums[3] = 40;
nums[4] = 50;
nums[5] = 60;

for (int i = 0; i < 6; i++) {
    printf("%d\n", nums[i]);
}

free(nums);
```

¿Qué hace internamente `realloc` ?

Puede pasar una de dos cosas:

Caso 1: puede ampliar la memoria en el mismo sitio

</> C



Caso 2: no puede ampliarla ahí

Entonces busca otra zona más grande, copia los datos antiguos y libera la anterior.

antes:

nums ----> [10][20][30]

después:

nums ----> [10][20][30][][][]

Pero quizá en otra dirección.

Por eso no debes guardar punteros internos antiguos.

Ejemplo peligroso:

```
int *nums = malloc(3 * sizeof(int));
int *p = &nums[1];

int *temp = realloc(nums, 100 * sizeof(int));
nums = temp;

*p = 50; // peligroso, p puede apuntar a memoria antigua
```

6. strdup

`strdup` duplica una cadena reservando memoria dinámica.

Ejemplo:

`</>` C



Eso crea una copia dinámica de "Diego" .

Equivale aproximadamente a:

```
char *nombre = malloc(strlen("Diego") + 1);
strcpy(nombre, "Diego");
```



El `+1` es por el carácter final `'\0'` .

Ejemplo

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    char *nombre = strdup("Diego");

    if (nombre == NULL) {
        return 1;
    }

    printf("%s\n", nombre);

    free(nombre);

    return 0;
}
```

Detalle importante

`strdup` reserva memoria, así que también se libera con:

```
</> C
```



Aunque uses cadenas, sigue siendo memoria dinámica.

7. Bloques de memoria

Ahora pasamos al segundo subapartado:

```
memcpy  
memset  
memcmp
```

Estas funciones trabajan con memoria en bruto.

No trabajan necesariamente con `int`, `char`, `struct`, etc. Trabajan con bloques de bytes.

Están en:

```
</> C
```



8. memset

`memset` rellena una zona de memoria con un valor byte a byte.

Sintaxis:

```
</> C
```



Ejemplo:

```
int nums[5];

memset(nums, 0, 5 * sizeof(int));
```

Esto pone todos los bytes del array a 0 .

Para enteros, poner todos los bytes a cero normalmente equivale a enteros con valor 0 .

Ejemplo con chars

```
char texto[10];

memset(texto, 'A', 9);
texto[9] = '\0';

printf("%s\n", texto);
```

Resultado:

```
</> C
```



Cuidado con memset en enteros

Esto parece lógico:

```
int nums[5];

memset(nums, 1, 5 * sizeof(int));
```

Pero no pone cada entero a 1 .

Pone cada byte a 1 .

Si un `int` ocupa 4 bytes, cada entero queda así:

</> C



Eso no es el número 1. Es otro valor.

Para inicializar enteros a 1, usa un bucle:

```
for (int i = 0; i < 5; i++) {  
    nums[i] = 1;  
}
```

9. memcpy

`memcpy` copia bytes de una zona de memoria a otra.

Sintaxis:

</> C



Ejemplo:

```
int origen[3] = {10, 20, 30};  
int destino[3];  
  
memcpy(destino, origen, 3 * sizeof(int));
```

Ahora `destino` contiene:

</> C



Ejemplo con estructuras

```
typedef struct {  
    int id;  
    char nombre[20];  
} Persona;  
  
Persona p1 = {1, "Ana"};  
Persona p2;  
  
memcpy(&p2, &p1, sizeof(Persona));
```

Ahora `p2` es una copia byte a byte de `p1`.

Cuidado con memoria solapada

`memcpy` no debe usarse si origen y destino se solapan.

Ejemplo peligroso:

```
char texto[] = "abcdef";  
  
memcpy(texto + 1, texto, 3);
```

Aquí origen y destino están en el mismo array y se pisan.

Para memoria solapada existe:

</> C



Aunque no aparece en tu apartado, es bueno conocerlo.

10. memcmp

`memcmp` compara dos bloques de memoria byte a byte.

Sintaxis:

<> C



Devuelve:

```
0   si son iguales
<0  si el primer bloque es menor
>0  si el primer bloque es mayor
```

Ejemplo:

```
int a[3] = {1, 2, 3};
int b[3] = {1, 2, 3};

if (memcmp(a, b, 3 * sizeof(int)) == 0) {
    printf("Son iguales\n");
}
```

Cuidado con estructuras

Esto puede ser peligroso:

```
typedef struct {
    char c;
    int x;
} Dato;
```

Una estructura puede tener bytes de relleno internos, llamados padding.

Entonces dos estructuras con los mismos campos podrían no tener exactamente los mismos bytes.

Por eso, para estructuras, muchas veces es mejor comparar campo a campo:

```
if (d1.c == d2.c && d1.x == d2.x) {  
    printf("Iguales\n");  
}
```

Código usando todo el apartado 10

Este programa usa:

```
malloc  
calloc  
sizeof  
realloc  
strdup  
free  
memset  
memcpy  
memcmp
```

```
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
  
typedef struct {  
    int id;  
    char *nombre;  
    int edad;  
} Persona;  
  
int main() {  
    int n = 3;  
  
    // malloc + sizeof  
    int *numeros = malloc(n * sizeof(int));  
  
    if (numeros == NULL) {  
        printf("Error con malloc\n");  
        return 1;  
    }  
}
```

```
for (int i = 0; i < n; i++) {
    numeros[i] = (i + 1) * 10;
}

printf("Array creado con malloc:\n");
for (int i = 0; i < n; i++) {
    printf("%d ", numeros[i]);
}
printf("\n\n");

// calloc
int *ceros = calloc(n, sizeof(int));

if (ceros == NULL) {
    printf("Error con calloc\n");
    free(numeros);
    return 1;
}

printf("Array creado con calloc:\n");
for (int i = 0; i < n; i++) {
    printf("%d ", ceros[i]);
}
printf("\n\n");

// realloc
int nuevoTam = 5;
int *temp = realloc(numeros, nuevoTam * sizeof(int));

if (temp == NULL) {
    printf("Error con realloc\n");
    free(numeros);
    free(ceros);
    return 1;
}

numeros = temp;

numeros[3] = 40;
numeros[4] = 50;

printf("Array después de realloc:\n");
for (int i = 0; i < nuevoTam; i++) {
    printf("%d ", numeros[i]);
}
printf("\n\n");

// strdup
```

```
Persona p1;
p1.id = 1;
p1.nombre = strdup("Diego");
p1.edad = 20;

if (p1.nombre == NULL) {
    printf("Error con strdup\n");
    free(numeros);
    free(ceros);
    return 1;
}

printf("Persona p1:\n");
printf("ID: %d\n", p1.id);
printf("Nombre: %s\n", p1.nombre);
printf("Edad: %d\n\n", p1.edad);

// memcpy
Persona p2;

memcpy(&p2, &p1, sizeof(Persona));

/*
    OJO:
    Esto copia también el puntero nombre.
    Es decir, p2.nombre apunta al mismo sitio que p1.nombre.

    Por eso NO haremos free(p2.nombre),
    porque sería un double free.
*/

printf("Persona p2 copiada con memcpy:\n");
printf("ID: %d\n", p2.id);
printf("Nombre: %s\n", p2.nombre);
printf("Edad: %d\n\n", p2.edad);

// memcmp
if (memcmp(numeros, ceros, n * sizeof(int)) == 0) {
    printf("Los primeros elementos de numeros y ceros son iguales\n");
} else {
    printf("Los primeros elementos de numeros y ceros son diferentes\n");
}

// memset
memset(ceros, 0, n * sizeof(int));

printf("\nArray ceros después de memset:\n");
for (int i = 0; i < n; i++) {
    printf("%d ", ceros[i]);
}
```

```
}  
printf("\n");  
  
// free  
free(numeros);  
free(ceros);  
free(p1.nombre);  
  
numeros = NULL;  
ceros = NULL;  
p1.nombre = NULL;  
p2.nombre = NULL;  
  
return 0;  
}
```

Compilación:

</> Bash



```
gcc programa.c -o programa
```

Ejecución:

</> Bash



```
./programa
```

En Windows:

</> Bash



```
programa.exe
```



Fuentes